PRODUCT REQUIREMENTS DOCUMENT (PRD)

RAGNext: Autonomous Version-Aware Document Intelligence Engine with Verifiable Grounding

Product Name	RAGNext (Cyber Knights Document Intelligence)	**Problem Statement ID**	SH-SVA-04 (Version-Aware Smart Virtual Assistant)
Authors / Team	Sainath Loganathan, Tahir Azeem (Cyber Knights)	**Target Release**	Stage 2 Evaluation & Enterprise Air-Gapped MVP

1. Executive Summary & Problem Scope Boundaries

Core Principle: *'Find the right information, from the right document version, with verifiable evidence.'*
Conventional vector RAG systems treat all document chunks equally and retrieve answers based strictly on semantic cosine distance. When policies undergo revisions (e.g. Leave Policy v1.0 ➔ v2.0 ➔ v3.0), older superseded clauses frequently score higher similarity than active ones due to exact keyword density. RAGNext eliminates this vulnerability by binding temporal lifecycle states to document embeddings and enforcing pre-retrieval policy filters.

Scope Dimension	In-Scope for Release v1.0 (Hackathon MVP)	Out-of-Scope (Future Enterprise Roadmap)
Document Formats	PDF (page-aware text extraction), DOCX (mammoth), TXT, and Markdown up to 25 MB.	Scanned raster images requiring heavy OCR pipelines; audio/video transcripts.
Lifecycle States	ACTIVE, SUPERSEDED, DRAFT, ARCHIVED with automatic lineage state transitions.	Git-style three-way branch merging of conflicting document text.
Retrieval Logic	Pre-retrieval temporal filtering, historical version routing, and conflict sentinel detection.	Cross-lingual translation of foreign language legal policies.
Inference & Storage	100% offline local inference via Ollama (llama3.2 + nomic-embed-text), SQLite/Postgres.	Cloud-hosted commercial AI APIs (OpenAI/Anthropic) to ensure air-gapped sovereignty.

2. Decoupled 3-Tier System Architecture & Technology Stack

Architecture Layer	Technology Choice	Technical Role & Rationale
Presentation Tier	React 18 + Vite + Bootstrap 5 + Lucide Icons	Snappy Virtual DOM updates, responsive grid, real-time citation cards, and visual revision trees.
API Gateway & Logic	Node.js 18+ / Express.js (ES Modules)	Asynchronous non-blocking I/O for concurrent document uploads, chunking, and REST query routing.
Document Ingestion	pdf-parse (page hook) + mammoth (DOCX) + Multer	Page-coordinate extraction, cryptographic upload hashing (`doc__`), and traversal defense.
Chunking Engine	Context-Preserving Semantic Chunker	Splits text into 500-char segments with 100-char overlap, attaching source page number and version ID.
Metadata Database	PostgreSQL Adapter + Local Persistent Store Fallback	Relational integrity for revision trees; automatic zero-friction fallback to disk JSON/SQLite for hackathons.
Vector Store	Embedded Persistent Vector DB (Chroma-spec)	In-memory cosine ranking with disk persistence; supports hard metadata filtering on version lifecycle.
AI Inference Engine	Ollama (llama3.2) + nomic-embed-text	100% offline air-gapped dense embeddings and chat synthesis with zero cloud telemetry or SaaS fees.
Fail-Safe Fallback	Deterministic Grounded Extractive Synthesizer	Guarantees 100% service uptime: deterministically extracts verified excerpts if LLM is offline.

3. Functional Requirements (MoSCoW Matrix)

ID	Pri.	Requirement Title	Detailed Acceptance Criteria
FR-01	**MUST**	**Multi-Format Ingestion**	Parses PDF, DOCX, TXT files up to 25MB; generates SHA-256 hash and sanitizes filename safely.
FR-02	**MUST**	**Document Grouping**	Organizes revisions into logical parent groups (e.g. 'Leave Policy' ➔ v1.0, v2.0, v3.0).
FR-03	**MUST**	**SemVer & Lifecycle**	Parses SemVer tags; automatically transitions older active versions to SUPERSEDED when promoted.
FR-04	**MUST**	**Page-Aware Chunking**	Splits documents into 500-char chunks with 100-char overlap, binding exact page numbers to metadata.
FR-05	**MUST**	**Local Vector Embeddings**	Computes 768-dim embeddings via nomic-embed-text; fallback to deterministic 128-dim subword vectors.
FR-06	**MUST**	**Pre-Retrieval Filter**	Restricts general policy search strictly to `status = 'ACTIVE'`, blocking superseded chunks completely.
FR-07	**MUST**	**Historical Query Routing**	Identifies version intent (e.g. 'in v1.0') and routes retrieval to that version with a warning badge.
FR-08	**MUST**	**Verifiable Citations**	Every answer must output document name, version tag, page number, and verbatim excerpt.
FR-09	**MUST**	**Conflict Sentinel**	Flags a prominent alert when multiple active policies coexist in the same group, preventing silent blends.
FR-10	**MUST**	**Missing Evidence Gate**	Returns explicit insufficient evidence status if similarity score < 0.45 or query is unanswerable.
FR-11	**SHOULD**	**1-Click Seed Dataset**	Dashboard button instantly loads Leave Policy and Remote Security Policy revision test suites.
FR-12	**SHOULD**	**Audit Telemetry Log**	Stores complete query string, cited version IDs, matched chunk IDs, latency, and timestamp.
FR-13	**COULD**	**Visual Version Diff**	Interactive UI diff showing clause additions, removals, and changes between two revisions.

4. Non-Functional Engineering Requirements (NFR)

ID	NFR Category	Engineering Target Metric	Verification & Validation Procedure
NFR-01	**Inference Latency**	< 2.5s (Extractive); < 4.0s (Ollama)	Benchmarked on standard developer CPU/GPU across 50 consecutive queries.
NFR-02	**Retrieval Speed**	< 150 ms across 10,000 chunks	In-memory vector cosine distance timing tracked via backend performance middleware.
NFR-03	**Air-Gapped Security**	Zero External Cloud Data Outbound	Packet monitoring confirming zero outbound HTTP requests to external AI APIs.
NFR-04	**High Availability**	100% Service Uptime Fallback	Automatic fallback to deterministic extractive synthesizer if Ollama daemon halts.
NFR-05	**Storage Sanitization**	Zero Path Traversal / MIME-sniffed	Automated path sanitization tests preventing `../` directory injection attacks.
NFR-06	**Evidence Grounding**	> 95% Verified Citation Accuracy	Factuality benchmark comparing synthesized claims to verbatim source document excerpts.

5. Relational Data Model & Storage Schema

Entity Table	Key Fields & Types	Relational Integrity & Lifecycle Constraints
document_groups	id (UUID), name (TEXT), category (TEXT), description, created_at	Parent entity for version lineages; cascades deletion to versions.
document_versions	id (UUID), group_id (FK), version_string, status, effective_date, file_path, checksum	Enforces lifecycle status ('ACTIVE', 'SUPERSEDED', 'DRAFT', 'ARCHIVED'); SHA-256 integrity.
document_chunks	id (UUID), version_id (FK), chunk_index (INT), page_number (INT), content (TEXT), embedding	Stores exact document page numbers and chunk offsets for verifiable citations.
audit_logs	id (UUID), query_text, cited_version_id, confidence, engine_used, latency_ms, timestamp	Tamper-evident query telemetry for ISO 27001 and NIST SP 800-53 audit compliance.

6. REST API Endpoints Specification

HTTP Method & Route	Request Payload / Headers	Response Behavior & Output Codes
POST /api/documents/upload	Multipart: file, group_id, version_string, status, effective_date	201 Created: Sanitizes file, chunks text, generates embeddings, updates lineage tree.
GET /api/documents	Query params: category, search (Optional)	200 OK: Returns list of document groups with active version number and total counts.
GET /api/documents/:id/versions	Path: Document Group ID	200 OK: Returns complete parent-child version lineage tree with lifecycle statuses.
POST /api/chat/query	JSON: { query: string, document_id?: string }	200 OK: Classifies intent, filters active version, generates answer with page citations.
POST /api/demo/seed	Empty body (Trigger)	200 OK: Loads Leave Policy and Remote Security Policy 3-version demo datasets.
GET /api/health	Empty body	200 OK: Reports connection status for PostgreSQL, SQLite fallback, and local Ollama.

7. Automated Test Suite & Hackathon Verification Matrix

Test Case #	Verification Scenario	Input Query / Action	Pass / Fail Criteria
TEST-01	**Metadata Ingestion**	Upload 3-version Leave Policy	PASS: Creates v1.0, v2.0, v3.0 parent-child lineage in metadata DB.
TEST-02	**Active Version Query**	'What is the annual leave allowance?'	PASS: Returns 20 days citing v3.0 [Active], Page 1. Rejects v1.0/v2.0.
TEST-03	**Historical Query**	'What was the allowance in version 2?'	PASS: Returns 18 days citing v2.0 with [Historical Record] banner.
TEST-04	**Missing Evidence Gate**	'Where is maternity leave explained?'	PASS: Returns explicit Insufficient Evidence status without hallucinating.
TEST-05	**Conflict Sentinel Alert**	Simulate two concurrent active versions	PASS: Dispatches live Conflict Sentinel alert, halting single answer.
TEST-06	**Deletion Cleanup**	Delete document group	PASS: Cascades deletion across metadata DB and in-memory vector store.